

TD 2 à 5 simulateur de pollution atmosphérique

Contexte : il est aujourd'hui primordial de contrôler la quantité de polluants dans l'air de nos grandes agglomérations ainsi que son évolution. Il est possible de prédire cette quantité par des simulations particulières où l'on pourra suivre la dispersion des particules de polluants dans l'air.

Nous proposons d'utiliser ce contexte comme fil rouge de nos séances de TD.

TD 5 Optimisation

5.1 Analyse de performances

Afin de mesurer les performances de votre code, vous construirez une classe `Timer` qui grâce à des méthodes `start`, `stop` et `print` vous permettra de calculer le temps passé entre les appels à `start` et `stop` et de l'afficher avec la méthode `print`. A l'intérieur de cette classe vous utiliserez les outils du standard `std::chrono` pour effectuer ces mesures. Vous pourrez par exemple stocker dans la classe deux `time_point`, l'un affecté lors du `start` et l'autre lors du `stop`. Vous créerez deux `Timer`, l'un pour calculer le temps pris par la simulation stationnaire et l'autre pour calculer le temps pris pour la simulation instationnaire. Vous afficherez ces résultats à l'écran en millisecondes. Ces mesures doivent être faites en faisant tourner le code en mode **Release**. Vous pourrez laisser CMake choisir les options de compilation adaptées, il vous suffira alors de le lancer avec l'option : `cmake -DCMAKE_BUILD_TYPE=Release`. Il vous faudra également désactiver tous les affichages à l'écran et dans les fichiers que vous pouviez avoir mis pour le debug et qui sont très consommateurs de temps CPU. Afin de pouvoir faire des mesures significatives, vous réaliserez un calcul pour un nombre de particules de l'ordre 10^6 . Afin d'équilibrer la charge de calcul entre le stationnaire et l'instationnaire, vous pourrez mettre trois fois plus de particules pour le cas stationnaire.

5.2 Amélioration des performances via `std::thread`

Un premier travail d'optimisation à grain grossier pourra consister à exécuter dans des threads distincts les différents noyaux de calcul de votre applications. Les phase d'`init` et de `compute` ne sont pas parallélisables entre elles en revanche vous allez pouvoir séparer dans des threads différents les deux populations de particules introduites au TD 4 pour simuler les différents modèles de vitesse de l'air (4.2). Afin de pouvoir comparer vos performances séquentielles et parallèles vous déplacerez le contenu de votre `main` au sein d'une fonction `solve()`. Vous créerez ensuite une nouvelle fonction `solve_parallel()` dans laquelle vous recopierez le contenu de `solve()`. C'est dans cette nouvelle méthode que vous ferez la parallélisation. Pour mieux encapsuler vous pouvez aussi décider de créer une nouvelle classe `Problem` qui contiendra ces deux méthodes `Problem::solve()` et `Problem::solve_parallel()`.

5.2.1 Premier exemple d'utilisation de `std::thread`

Depuis C++11, il est assez facile d'introduire des processus légers (threads). En effet vous avez accès à la classe `thread` depuis le header `<thread>`. Cette classe prend à la construction une λ -fonction qui sera exécutée par le processus. Ainsi, l'exemple de code suivant exécutera en parallèle deux threads :

```
#include <thread>

int main(){
    std::thread first_thread([](){
        // do something : compute a first particle group
    });
    std::thread second_thread([](){
        // do something : compute a second particle group
    });
    first_thread.join();
    second_thread.join();
    return 0;
}
```

Pour pouvoir exécuter un programme avec des `std::thread` il faudra embarquer la bibliothèque de threads de votre système d'exploitation à l'édition des liens. CMake est là pour vous aider dans ce travail : il faudra ajouter les lignes suivantes dans votre CMakeLists.txt, en remplaçant `MyExe` par le nom de la target CMake associée à votre exécutable :

```
find_package(Threads)
target_link_libraries(MyExe Threads::Threads)
```

5.2.2 Mise en oeuvre dans le simulateur

Comme vous le voyez dans l'exemple, les opérations effectuées dans le thread, sont passées sous la forme d'une `$\lambda$`-fonction. Vous pouvez accéder à la mémoire du thread principal en capturant des variables dans les `$\lambda$` mais attention aux accès concurrents. Dans notre exemple, vous pourrez créer plusieurs groupes de particules, par exemple pour des modèles de vitesses différents et faire l'initialisation, le calcul et l'affichage dans un thread différent pour chaque groupe de particules.

Vous comparerez ensuite les temps de calcul au temps séquentiel, toujours en mode Release.

5.3 Amélioration des performances via une parallélisation des boucles

Les gains possibles avec cette approche à grains grossiers sont relativement limités : ils ne permettent pas de paralléliser pour un groupe de particules données. Or les boucles de calculs sur les particules sont *embarrassingly parallel*. En effet les calculs sont complètement indépendants pour chaque particule. Nous allons donc maintenant réaliser un parallélisme à grain plus fin, au niveau des boucles de calcul. Le plus simple pour paralléliser nos boucles tout en restant sur du C++ standard sera d'utiliser les algorithmes parallèles de la STL. En effet dans les parties 3.5.2 du TD3 et 4.1 du TD4, vous avez introduit des algorithmes de la STL dans la totalité des boucles sur le nuage de particules (initialisation et calcul). Nous allons pouvoir introduire assez simplement du parallélisme.

5.3.1 Préparation

Pour pouvoir faire fonctionner les algorithmes parallèles de la STL, il est nécessaire d'embarquer Intel TBB à l'édition des liens. Pour l'installer sur votre machine Linux, vous pouvez le faire via le paquet :

```
sudo apt install libtbb-dev
```

Il faudra donc modifier votre CMakeLists.txt, comme d'habitude en remplaçant `MyExe` par le nom de la target CMake associée à votre exécutable :

```
find_package(Threads)
find_package(TBB)
target_link_libraries(MyExe Threads::Threads TBB::tbb)
```

5.3.2 Mise en oeuvre

La parallélisation de votre algorithme se fera maintenant très simplement, en ajoutant en paramètre l'environnement d'exécution :

```
#include <execution>
#include <algorithm>
int main() {
    std::for_each(std::execution::par, times.begin(), times.end(), [](auto time) {
        // your function
    });
}
```

Vous pourrez regarder sur https://en.cppreference.com/w/cpp/algorithm/execution_policy_tag_t le sens des différentes valeurs dans `std::execution` pour choisir la plus adaptée. Vous pourrez comparer avec le séquentiel grâce à la valeur `std::execution::seq`.

Vous utiliserez cette syntaxe dans toutes les boucles sur votre nuage de particules, que ce soit pour l'initialisation ou pour le calcul des positions et des vitesses.

Bonus remplacer `std::thread` par `std::async`

L'utilisation de `std::async` peut être assez intéressante. En effet cette fonction va créer le thread pour vous, encapsuler son éventuel résultat dans une classe `std::future`, et vous permettre de choisir entre une exécution séquentielle ou parallèle. Si nous reprenons l'exemple donné en 5.2.1 et que nous le réécrivons à base d'`async` cela donnera :

```
#include <future>

int main(){
    std::future first_task = std::async(std::launch::async, [](){
        // do something : compute a first particle group
    });
    std::future second_task = std::async(std::launch::async, [](){
        // do something : compute a second particle group
    });
}
```

```
});  
first_task.wait();  
second_task.wait();  
return 0;  
}
```

Il est également possible de mettre comme premier argument `std::launch::deferred` ; il n'y aura alors pas de création de thread et la tâche sera exécutée séquentiellement à l'appel du `wait()`. Vous pourrez alors facilement comparer les performances en séquentiel ou parallèle. Vous pourrez aussi comparer les performances de la solution à base d'`async` avec celles de la solution à base de `thread`.